Min Height Bst

AlgoExpert · Medium · Binary Search Trees

The problem

- Build a BST of minimal height from a sorted list of distinct ints (pick middle as root).

Intuition

- Choosing the middle element as the root keeps both halves balanced.

Brute force vs optimal

- Insert in sorted order → a linked-list-shaped tree, height n
- Insert values one by one in a smart order → O(n log n)
- Build directly from midpoints → O(n), height $\lceil \log_2(n+1) \rceil$

Approach

- 1. Take the middle of the sorted range as the root
- 2. Recurse on the left half
- 3. Recurse on the right half

Worked example





Range [0, 8] → root 10

Worked example





Range [0, 0] → root 1

Worked example

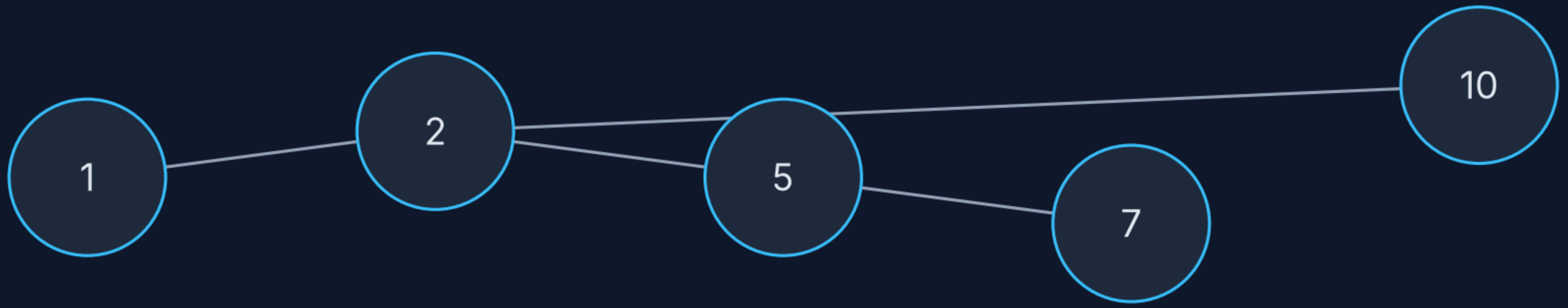




Range [2, 3] → root 5

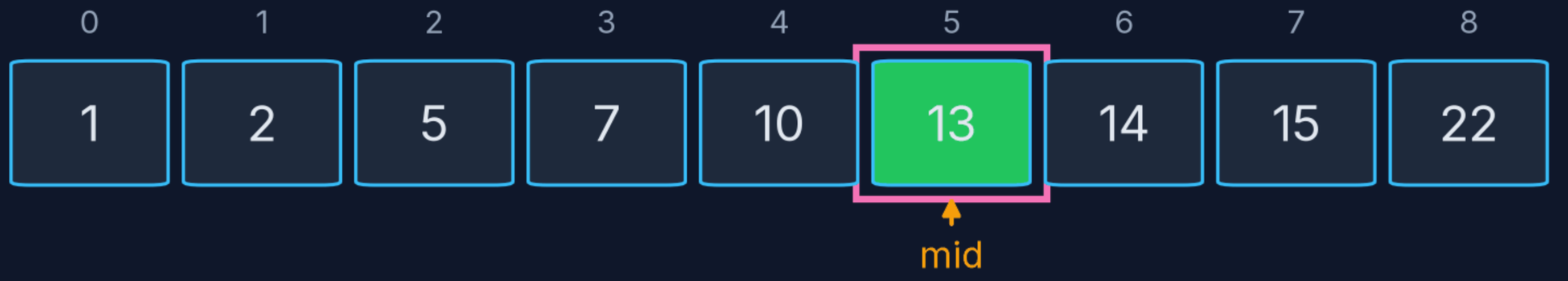
Worked example

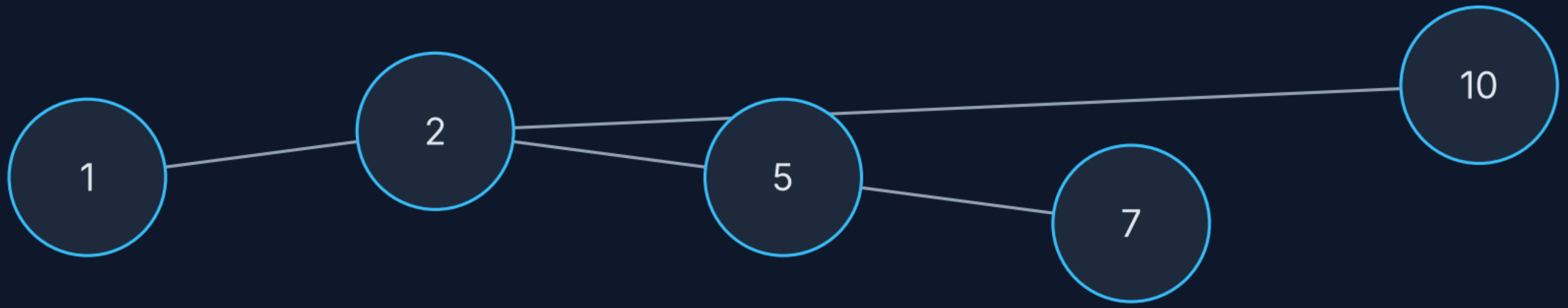




Range [5, 8] → root 14

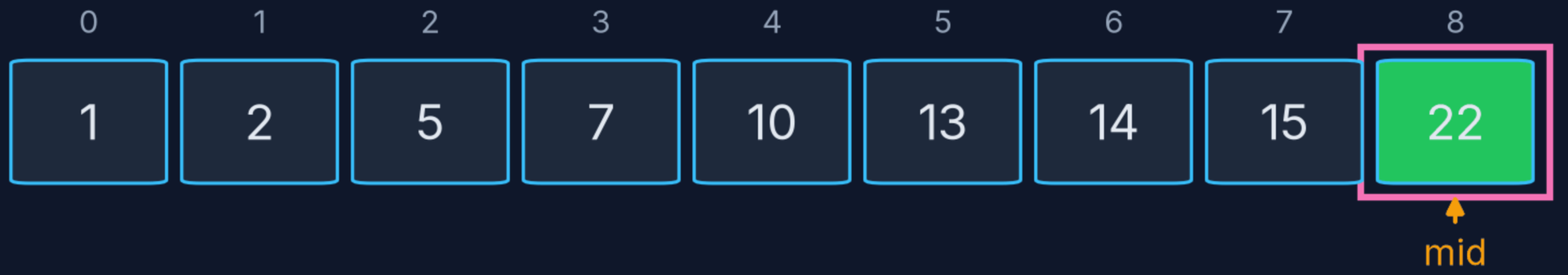
Worked example

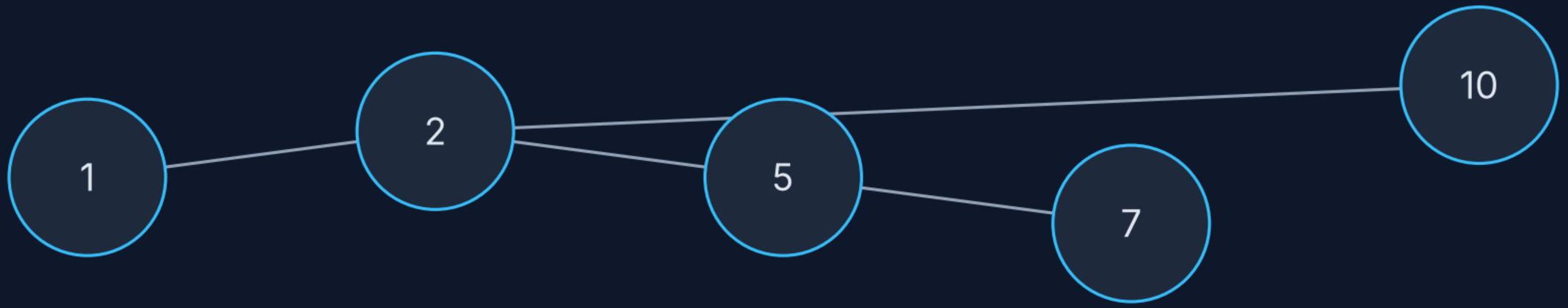




Range [5, 5] → root 13

Worked example





Range [8, 8] → root 22

Java solution

```java
public class MinHeightBst {
    public static BST minHeightBst(List<Integer> a) { return build(a, 0, a.size() - 1); }

    private static BST build(List<Integer> a, int lo, int hi) {
        if (lo > hi) return null;
        int mid = (lo + hi) / 2;
        BST n = new BST(a.get(mid));
        n.left = build(a, lo, mid - 1);
        n.right = build(a, mid + 1, hi);
        return n;
    }

    public static int height(BST t) { return t == null ? 0 : 1 + Math.max(height(t.left), height(t.right)); }
}
```

Complexity

- Time: O(n)
- Space: O(n)

Edge cases & pitfalls

- Even-length ranges: either middle works
- Empty array → null
- Build nodes directly; don't call insert (O(n log n))

Interview tips & follow-ups

- Prove the height is minimal
- Follow-up: sorted linked list → BST in O(n)

Recap

- Choosing the middle element as the root keeps both halves balanced.
- Complexity: O(n)